RESPONSIVE IMAGES
+ CSS CLAMP(), MIN() & MAX()

A beginner-friendly guide to making images and layouts adapt to different screen sizes

What you will learn

- How responsive images work and why they matter.
- How to make images shrink, grow, crop, and keep their proportions.
- How to use the HTML picture, source, srcset, and sizes elements/attributes.
- How CSS clamp(), min(), and max() work, with practical examples.
- How to combine these techniques in a small responsive webpage.

Learning approach: Start with the simple CSS rules, understand each value, then practise with the complete examples. You can use these techniques in your Stressfree shortlet website, including its hero section, apartment cards, and gallery.

Prepared as a step-by-step study note • HTML + CSS

1. What does responsive mean?

A responsive website adjusts its layout and content to fit the screen being used. A desktop monitor is wide; a phone is narrow; a tablet is somewhere in between. A responsive image changes size to fit its available space instead of overflowing, becoming distorted, or forcing the visitor to scroll sideways.

Think of an image as a guest in a container: the image should fit the space available while keeping its shape unless you intentionally crop it.

Why responsive images matter

- They prevent images from overflowing on small screens.
- They preserve the image's natural proportions and avoid stretching.
- They make pages easier to use on phones, tablets, and desktops.
- They can improve loading performance when the browser is offered suitable image sizes.

2. The most important responsive image rule

For many normal website images, begin with this CSS:

```
img {
  max-width: 100%;
  height: auto;
  display: block;
}
```

max-width: 100%; means the image is allowed to become smaller, but it cannot become wider than its parent container. If the image's original width is smaller than the container, it does not automatically stretch to fill it.

height: auto; lets the browser calculate the height from the image's original proportions. This prevents a photo from looking squashed or stretched.

display: block; makes the image behave as a block element. This is useful for removing the small baseline gap that can appear under inline images.

Example: if a 1200px-wide image sits inside a 360px phone-sized container, max-width: 100% lets it display at 360px wide. Its height is reduced proportionally.

A complete starter example

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Responsive image practice</title>
  <style>
    .photo-box {
      width: 90%;
      max-width: 700px;
      margin: 30px auto;
    }

    .photo-box img {
      max-width: 100%;
      height: auto;
      display: block;
    }
  </style>
</head>
<body>
  <div class="photo-box">
    <img src="apartment.jpg" alt="A bright shortlet apartment">
  </div>
</body>
```

```
</html>
```

The viewport meta tag is important: it tells mobile browsers to use the device's actual layout width rather than pretending the page is a much wider desktop page.

3. Image sizing: width, max-width, and height

width: 100%

The image takes the full width of its parent. It can grow or shrink to match that parent.

```
img {
  width: 100%;
  height: auto;
}
```

max-width: 100%

The image can shrink when necessary, but it will not grow beyond its natural rendered size just because the parent is wider. This is a good general-purpose default.

```
img {
  max-width: 100%;
  height: auto;
}
```

width with max-width

You can give an image a preferred width while setting a maximum limit:

```
.apartment-photo {
  width: 700px;
  max-width: 100%;
  height: auto;
}
```

On a wide screen it can be up to 700px. On a narrow screen it shrinks to fit the parent.

object-fit: cover

Sometimes you want every card photo to have the same shape. object-fit: cover fills the image box and crops the extra parts. It does not squeeze the image.

```
.apartment-card img {
  width: 100%;
  height: 220px;
  object-fit: cover;
  object-position: center;
}
```

cover: fills the box, possibly cropping edges. **contain:** shows the whole image, possibly leaving empty space. **object-position:** controls which part stays visible, such as center, top, or 60% 30%.

4. Responsive image layouts with CSS

A responsive gallery using CSS Grid

```
.gallery {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 16px;
}

.gallery img {
  width: 100%;
  height: 200px;
  object-fit: cover;
}
```

```
/* On smaller screens, use fewer columns */
@media (max-width: 700px) {
  .gallery {
    grid-template-columns: repeat(2, 1fr);
  }
}

@media (max-width: 450px) {
  .gallery {
    grid-template-columns: 1fr;
  }
}
```

The media queries change the number of columns at chosen screen widths. The images then fill the width of their grid cells.

A flexible gallery without fixed breakpoints

```
.gallery {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 16px;
}
```

This tells Grid to fit as many columns as it can, with each column at least 220px wide and otherwise sharing the available space. The browser automatically moves cards to new rows when space is limited.

5. HTML responsive images: picture, srcset, and sizes

CSS makes an image fit its layout. HTML responsive-image features can also let the browser choose a suitable image file for the device or screen conditions. This is useful when you have multiple versions of the same photo.

The picture element: art direction

Use picture when you want to show a different crop or composition at a particular screen size. For example, a wide landscape photo on desktop and a portrait crop on mobile.

```
<picture>
  <source
    media="(max-width: 600px)"
    srcset="apartment-portrait.jpg">
  <img
    src="apartment-landscape.jpg"
    alt="Modern shortlet apartment living room">
</picture>
```

The browser checks the source conditions. On a screen 600px wide or less, it can use the portrait file. Otherwise, it uses the img fallback. Keep the img element: it provides the fallback and the alt text.

srcset and sizes: different resolutions

```
<img
  src="room-800.jpg"
  srcset="
    room-400.jpg 400w,
    room-800.jpg 800w,
    room-1200.jpg 1200w"
  sizes="(max-width: 600px) 100vw, 50vw"
  alt="A neatly furnished apartment bedroom">
```

srcset lists available image files and their intrinsic widths. The w descriptor tells the browser the width of each file. **sizes** describes how wide the image is expected to appear in the layout at different viewport widths. The browser uses this information, along with device pixel density and other factors, to select a file.

> Do not confuse srcset with CSS width. srcset helps select which file to download; CSS controls how the selected image is displayed.

6. What is clamp()?

clamp() is a CSS function that sets a value with a minimum, a preferred or flexible value, and a maximum. It is written like this:

```
property: clamp(MINIMUM, PREFERRED VALUE, MAXIMUM);
```

Read it as: 'Use the preferred value when possible, but never go below the minimum or above the maximum.'

Example: responsive heading

```
.hero-title {
  font-size: clamp(2rem, 5vw, 4rem);
}
```

The three parts:

- **2rem** — the smallest allowed font size.
- **5vw** — the flexible preferred size. vw means 1% of the viewport width.
- **4rem** — the largest allowed font size.

On a narrow screen, 5vw may be smaller than 2rem, so the minimum wins. On a wide screen, 5vw may exceed 4rem, so the maximum wins. Between those points, the value can scale smoothly.

clamp() for image width

```
.hero-image {
  width: clamp(280px, 70vw, 900px);
  max-width: 100%;
  height: auto;
  display: block;
  margin-inline: auto;
}
```

This asks for a width that grows with the viewport, but stays between 280px and 900px. max-width: 100% is an extra safeguard so the image does not overflow a parent narrower than 280px.

Important: clamp() does not make an image responsive by itself in every situation. It controls the CSS value you apply it to. You still need to think about the parent width, image proportions, and small-screen overflow.

7. What is min()?

min() returns the smallest of the values you provide. It is useful when you want an element to grow but never exceed a limit.

```
.content {
  width: min(90%, 1100px);
  margin-inline: auto;
}
```

The browser compares 90% of the available containing width with 1100px and uses whichever is smaller. On a small screen, 90% will usually be smaller. On a very wide screen, the width stops at 1100px.

min() for an image

```
.feature-image {
  width: min(100%, 850px);
  height: auto;
  display: block;
  margin-inline: auto;
}
```

The image can use all of the available width, but it will not exceed 850px.

8. What is max()?

max() returns the largest of the values you provide. It is useful when you want to guarantee a minimum size, while still allowing a larger value when available.

```
.panel {
  width: max(280px, 40%);
}
```

The browser compares 280px with 40% of the containing block and uses the larger value. Be careful: if the parent is narrower than 280px, this can cause horizontal overflow.

max() for spacing

```
.page-section {
  padding: max(20px, 4vw);
}
```

The section gets at least 20px of padding. On a wider viewport, 4vw may become larger and create more breathing room.

9. Comparing clamp(), min(), and max()

Function	What it does	Easy way to remember
min(A, B)	Chooses the smaller value	Never exceed the smaller limit
max(A, B)	Chooses the larger value	At least the larger requirement
clamp(MIN, FLUID, MAX)	Keeps a fluid value between two limits	Not too small, not too large

Quick examples:

```
/* Smaller of 90% or 1000px */
width: min(90%, 1000px);

/* Larger of 20px or 3vw */
padding: max(20px, 3vw);

/* Fluid size, limited at both ends */
font-size: clamp(1rem, 2vw, 1.5rem);
```

10. Build a responsive Stressfree hero section

This example combines a flexible content width, a responsive image, and clamp() for typography. Replace the image filename with one in your own project folder.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Stressfree | Your home away from home</title>
  <style>
    * {
      box-sizing: border-box;
    }

    body {
      margin: 0;
      font-family: Arial, sans-serif;
    }

    .hero {
      width: min(92%, 1100px);
      margin: 40px auto;
      text-align: center;
    }
```

```
    .hero h1 {
      font-size: clamp(2rem, 5vw, 4rem);
      margin-bottom: 12px;
    }

    .hero p {
      font-size: clamp(1rem, 2vw, 1.2rem);
      max-width: 650px;
      margin: 0 auto 24px;
    }

    .hero img {
      width: min(100%, 900px);
      height: clamp(220px, 45vw, 480px);
      object-fit: cover;
      object-position: center;
      border-radius: 18px;
      display: block;
      margin-inline: auto;
    }

    @media (max-width: 500px) {
      .hero {
        margin-block: 24px;
      }

      .hero img {
        border-radius: 10px;
      }
    }
  </style>
</head>
<body>
  <section class="hero">
    <h1>YOUR HOME AWAY FROM HOME</h1>
    <p>Enjoy a comfortable stay with Stressfree shortlet apartments.</p>
    <img src="Today1.jpg" alt="Comfortable shortlet apartment">
  </section>
</body>
</html>
```

Notice that clamp() is used for the heading, paragraph, and image height. min() limits the content and image widths. The image uses object-fit: cover because the design gives it a controlled height.

11. Common mistakes and how to fix them

The image stretches or looks squeezed. Use height: auto for naturally scaled images. If you set a fixed height, use object-fit: cover or contain.

The image is wider than the phone screen. Check that it has max-width: 100%, and that its parent does not have an unnecessarily large fixed width.

The image becomes too tiny. Check whether its parent is narrow, and whether you used width: 100% or an overly restrictive max-width.

clamp() seems not to change the value. Check the middle value and units. The minimum or maximum may be controlling the result.

max() causes sideways scrolling. A minimum such as 400px can be wider than a phone. Use min() or clamp() when the value must fit a narrow container.

The picture looks cropped. That is expected with object-fit: cover. Try object-fit: contain or adjust object-position.

The mobile image file never appears. Check the picture source media condition and file path. Remember that the img element is the fallback.

12. A simple testing routine

- Open the page in your browser and use Developer Tools' device toolbar.
- Test a narrow phone width, a larger phone, a tablet width, and a desktop width.
- Look for horizontal scrolling. If the page moves sideways, find the element wider than the viewport.
- Check that the image subject is still visible after cropping.
- Check text size and spacing; make sure headings do not become enormous or unreadably small.
- Test with a real phone when possible, not only the browser preview.

13. Practice exercises

Exercise 1 — Basic responsive image: Add an image to a page. Make it fill its parent but never overflow. Keep its natural proportions.

Exercise 2 — Apartment card: Create a card with an image that is 220px high and uses object-fit: cover. Make the card fit a one-column phone layout and a three-column desktop layout.

Exercise 3 — Clamp heading: Make a heading that starts at 24px, grows with the viewport, and never exceeds 56px. Try writing the clamp() expression yourself.

Exercise 4 — min and max: Give a content wrapper a width of the smaller of 92% and 1000px. Then give its padding at least 18px, or 3vw if that is larger.

Exercise 5 — Picture: Create a portrait crop for screens 600px wide or less, with a landscape image as the fallback.

Suggested answers:

```
/* Exercise 1 */
img { max-width: 100%; height: auto; }

/* Exercise 2 */
.card img {
  width: 100%;
  height: 220px;
  object-fit: cover;
}
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(230px, 1fr));
  gap: 16px;
}

/* Exercise 3: 24px = 1.5rem; 56px = 3.5rem */
h1 { font-size: clamp(1.5rem, 4vw, 3.5rem); }

/* Exercise 4 */
.wrapper {
  width: min(92%, 1000px);
  padding: max(18px, 3vw);
  margin-inline: auto;
}

/* Exercise 5 */
<picture>
  <source media="(max-width: 600px)" srcset="portrait.jpg">
  <img src="landscape.jpg" alt="Apartment interior">
</picture>
```

The final picture example is HTML, not CSS; place it inside the body of your page. The other examples belong in your stylesheet or a style element.

14. Quick revision sheet

- **img { max-width: 100%; height: auto; }** — reliable starting point for flexible images.

- **width: 100%;** — fill the parent width.
- **object-fit: cover;** — fill a fixed image box by cropping if needed.
- **object-fit: contain;** — show the whole image inside the box.
- **min(A, B)** — choose the smaller value.
- **max(A, B)** — choose the larger value.
- **clamp(MIN, FLUID, MAX)** — allow a value to scale within limits.
- **picture** — offer different image sources, often for different crops.
- **srcset + sizes** — offer image resolutions and explain expected display width.

Best beginner habit: Do not reach for clamp(), min(), or max() before understanding the simple width rules. First make the image fit with max-width: 100% and height: auto. Then use the CSS functions to control how the surrounding layout and dimensions scale.

End of notes.